AI 协作反思日志 — Phase 2 体系结构设计

团队：awencat 日期：2026.05.03

1. 本阶段 AI 使用清单

任务	AI工具	Prompt摘要	AI输出质量(1-5)	人工修改幅度(%)
架构风格候选方案分析	ChatGPT	对比MVC分层、微服务、事件驱动三种架构风格，结合本项目10周开发周期、4人团队的技术基础，从开发复杂度、可落地性、可扩展性等维度给出针对性分析，为架构选型提供初步依据。	4	20%
架构辩论赛4种Prompt咨询	ChatGPT	采用四种不同Prompt策略向AI咨询架构方案：直接推荐架构、提供项目全约束咨询、要求多架构方案对比、让AI扮演专业架构师分析，为团队架构辩论赛提供4组差异化素材，支撑辩论结论的形成。	4.5	15%
模块划分与接口定义	ChatGPT	基于确定的分层+MVC单体架构，结合校园二手交易平台的核心业务需求，划分用户、商品、订单等核心业务模块，明确各模块的核心职责、模块间依赖关系，初步梳理模块接口雏形。	4	25%
3条ADR编写（初稿）	ChatGPT	按照ADR标准格式，分别编写架构风格选择、技术栈选择、数据库选型3条ADR初稿，涵盖背景、决策、后果、替代方案等核心模块，为人工修订提供基础框架。	3.5	30%
架构设计文档整合	ChatGPT	整合架构概览、模块划分、技术选型、ADR集合等核心内容，按照规范的架构设计文档结构，生成完整的架构设计文档初稿，包含架构图、模块职责、技术栈详情等关键内容。	4	25%

2. AI 助力分析（Q1）

本阶段 AI 最大的贡献是：

- 快速提供多套架构候选方案及详细对比，清晰梳理各架构的优势与不足，有效降低架构选型的决策成本和时间成本；
- 通过四种不同Prompt策略生成差异化架构建议，为团队架构辩论赛提供丰富素材，帮助成员快速识别各架构与项目实际的适配度；
- 高效生成ADR和架构设计文档初稿，替代文档编写中重复性、框架性工作，提升团队整体开发效率；
- 让团队无需在冗余工作上消耗过多精力，可聚焦于核心决策、细节优化和业务落地。 不同 Prompt 策略下 AI 回答质量的差异：
- 直接询问“推荐什么架构”：回答偏通用化，仅罗列常见架构及基础特点，缺乏对本项目核心约束的针对性，回答深度不足，质量一般；
- 提供详细项目约束咨询：精准结合项目10周周期、4人团队、无微服务经验等实际情况，聚焦可落地性，建议贴合课程项目需求，针对性和实用性最强，质量最高；
- 要求多架构方案对比：全面覆盖开发复杂度、可维护性、可扩展性等核心维度，横向对比清晰，逻辑严谨，助力团队系统梳理候选方案，质量较好；
- 让AI扮演架构师角色：回答更具专业性和系统性，不仅给出架构建议，还分析决策权衡、潜在风险及未来演进路线，与带约束咨询质量持平，参考价值高。

3. AI 误导分析（Q2）

AI 在架构建议中的典型问题：

- 问题1：AI推荐微服务架构时，未充分考虑本项目核心约束。 具体：未兼顾课程项目10周短周期、4人团队无微服务开发经验的现状，过度强调微服务的可扩展性和先进性，忽略10周内的可落地性和团队学习成本，易导致项目无法按时交付； 如何发现：团队开展架构辩论赛时，对比4位成员的AI回答及项目实际目标，发现该建议与“10周内完成可运行原型”的核心目标严重脱节，执行难度极高； 如何修正：明确否决微服务架构，聚焦分层+MVC模式的模块化单体架构，模块划分时保留清晰边界，为未来扩展预留空间，确保项目按时落地。
- 问题2：AI生成ADR时，“后果”模块表述笼统，缺乏针对性和可操作性。 具体：风险分析为通用化表述，未结合校园二手交易平台具体业务场景，且无明确可操作的缓解措施，导致ADR实用性不足，无法指导

后续开发； 如何发现：人工修订ADR初稿时，发现风险描述未贴合商品发布、订单管理等核心业务，缓解措施模糊，无法落地执行； 如何修正：补充贴合项目场景的具体风险，如单体架构“大泥球”、数据库N+1查询等问题，同时制定可操作缓解措施，如规范代码、定期审查、优化索引等。
- 问题3：AI划分模块时，接口定义不清晰，缺乏具体规范。 具体：仅明确模块核心职责和大致依赖关系，未明确接口数据格式、调用方式和参数规范，影响后续前后端联调和开发效率； 如何发现：团队整合架构设计文档时，发现模块接口细节缺失，无法直接用于前后端对接和模块联调； 如何修正：人工补充各模块核心接口的调用方式、请求/响应格式、参数规范及权限要求，明确模块间通信标准，为后续开发提供依据。

4. Prompt 改进计划（Q3）

上阶段（P1）提出的改进措施，验证结果：

- 上阶段提出“明确Prompt中的项目约束和任务要求”，本阶段全面应用后效果显著；
- AI回答的针对性和可落地性大幅提升，无效信息减少，避免了通用化、不贴合项目实际的建议；
- 该改进措施切实有效，可继续沿用并进一步优化。 本阶段新的改进措施：

1. 编写Prompt时，明确要求AI结合“校园二手交易平台”具体业务场景（商品发布、订单管理等），避免通用化、模板化回答；
2. 生成ADR、模块设计等专业文档时，明确要求包含“具体风险+可操作缓解措施+场景案例”，提升文档实用性；
3. 咨询架构相关问题时，固定加入项目核心约束（课程项目、10周周期、4人团队等），避免AI过度设计建议；
4. 要求AI生成文档时，严格遵循指定格式、结构和命名规范，减少人工排版调整工作量。

5. 本阶段的核心工程判断

决策内容：

- 架构选型：采用“前后端分离 + 分层架构 + MVC模式的模块化单体架构”，不采用微服务、事件驱动架构作为主体架构；
- 局部借鉴：仅在订单消息通知、评价提醒等局部功能，借鉴事件驱动思想，简化开发复杂度；
- 技术栈：确定为Spring Boot + Vue.js + MySQL + MyBatis Plus，兼顾开发效率和可维护性；
- 数据库：采用MySQL关系型数据库，遵循规范化设计，合理设计表结构、索引和外键，同时预留反范式空间平衡性能与冗余。 为什么 AI 无法做这个决策：
- 无法综合权衡项目核心场景：AI可提供架构方案及技术层面分析，但无法结合课程项目、10周短周期、小团队等场景，判断过度设计对交付周期、协作效率的负面影响；
- 无法优先兼顾核心目标：AI无法优先选择“可落地性”而非“技术先进性”，无法根据“完成可运行原型、通过课程验收”的核心目标做出取舍；
- 缺乏工程实践经验：AI无法充分考量开发成本、团队学习成本、后续维护难度等实际因素；
- 需主观灵活取舍：核心工程判断需结合多维度综合权衡，需要人类主观判断，这是AI目前无法替代的，必须由团队决策。